```
            }
            delete old_head;    <-- 7
        }
        else
        {
            chain_pending_node(old_head);    <-- 8
            --threads_in_pop;
        }
    }
    void chain_pending_nodes(node* nodes)
    {
        node* last=nodes;
        while(node* const next=last->next)    <-- 9 Follow the next pointer chain to the end
        {
            last=next;
        }
        chain_pending_nodes(nodes,last);
    }
    void chain_pending_nodes(node* first,node* last)
    {
        last->next=to_be_deleted;    <-- 10
        while(!to_be_deleted.compare_exchange_weak(    <-- 11 Loop to guarantee that last->next is correct
                  last->next,first));
    }
    void chain_pending_node(node* n)
    {
        chain_pending_nodes(n,n);    <-- 12
    }
};
```

If the count of `threads_in_pop` is 1 when you're trying to reclaim the node ❶, you're the only thread currently in `pop()`, which means it's safe to delete the node you just removed ❼, and it *may* also be safe to delete the pending nodes. If the count is *not* 1, it's not safe to delete any nodes, so you have to add the node to the pending list ❽.

Assume for a moment that `threads_in_pop` is 1. You now need to try to reclaim the pending nodes; if you don't, they'll stay pending until you destroy the stack. To do this, you first claim the list for yourself with an atomic `exchange` operation ❷ and then decrement the count of `threads_in_pop` ❸. If the count is zero after the decrement, you know that no other thread can be accessing this list of pending nodes. There may be new pending nodes, but you're not bothered about them for now, as long